대시보드 증감 사유 분석 Tableau AI 에이전트

대시보드 숫자가 왜 오르고 내렸는지 자연어로 묻는 Tableau AI 에이전트. PoC와 사내 연구를 거쳐 본 프로젝트로 확정

2026.01 – 03, 2026.06 – 09 | 디포커스 | PoC·연구 설계 · 개발 | Tableau · LangGraph · PostgreSQL · pgvector · FastAPI
hoyoungpark.com/projects/tableau-ai-agent

한눈에 보기

기여

PoC 1인 설계·개발, 사내 연구 2명, 본 프로젝트 에이전트 파트

주요 업무

- PoC 대형 물류 기업(2026.01 – 03): Tableau MCP 대신 Metadata API·REST로 대시보드 구조와 값을 하루 1회 적재하고, langchain_tableau 없이 LangGraph(ReAct) 에이전트를 직접 구현
- 검색은 Qdrant 대신 고객사가 이미 쓰는 PostgreSQL에 pgvector로 통일
- 사내에 모아 둔 뉴스 데이터를 API로 연결해, 숫자가 변한 기간의 기사를 찾아 증감 사유로 붙이는 흐름 구현
- 사내 연구(2026.06 – 09, 2명): GraphRAG·멀티모달·워크북 XML 세 방식을 비교해 워크북 XML 구조 적재를 채택

결과

- PoC 고객사 임원 검토 통과, 사내 연구로 연결
- 본 프로젝트 착수 확정 (2026.12 – 2027.02), 목표는 증감 사유 분석 자동화

배경과 문제

현업은 대시보드 숫자가 왜 바뀌었는지를 엑셀로 따로 정리해 공유하고 있었습니다. 이 일을 AI에게 맡기려 하자 두 가지 간극이 드러났습니다. 하나는 사람은 화면을 보고 묻는데 AI는 데이터 소스만 본다는 것입니다. 화면의 숫자는 필터와 집계를 거친 값이라, 원본 데이터만 받아서는 같은 숫자가 나오지 않습니다. 다른 하나는 증감 사유 자체가 데이터 안에 없다는 것입니다. 현업 담당자는 수출량이 떨어지면 그 원인을 데이터가 아니라 바깥 소식에서 찾고 있었습니다.

내가 한 일

PoC를 혼자 설계·개발하고, 그 결과를 사내 연구로 이어 가 본 프로젝트 착수까지 연결했습니다. 본 프로젝트에서는 에이전트 파트를 맡습니다.

> **Tableau가 주는 도구로는 화면을 볼 수 없었다**
>
> Tableau가 제공하는 langchain_tableau는 tool 하나가 데이터 소스 하나에 묶이고, 호출마다 인증·메타데이터 조회·질의 작성을 다시 합니다. 데이터 소스 두 개에 걸친 질문을 못 받고, 원본에 직접 질의하니 화면의 필터·집계를 거친 숫자와도 맞지 않았습니다. Tableau MCP는 같은 정보를 REST·Metadata API로도 받을 수 있는데 tool로 그때그때 불러 느렸고, 화면 데이터와 데이터 소스 필드를 이어 줄 정보가 없어 서로 맞춰지지 않았습니다. 그래서 라이브러리를 버리고 API로 직접 받아 LangGraph 에이전트를 만들었습니다.

새 인프라 대신 고객사가 이미 쓰는 것 위에

고객사는 라이브 데이터가 없고 모든 데이터가 배치로 돕니다. 그래서 질문마다 Tableau를 부르는 대신 배치 주기에 맞춰 하루 1회 미리 적재했고, 이쪽이 더 빨랐습니다. 벡터 DB도 Qdrant를 새로 들이지 않고 이미 쓰는 PostgreSQL에 pgvector를 올렸습니다. 새 인프라를 깔지 않았기 때문에 고객사가 받아들일 수 있었습니다.

증감 사유는 데이터 밖에 있다

다들 숫자 변동을 LLM에게 그냥 물어 답하게 하려 했습니다. 하지만 현업 담당자는 수출량이 떨어지면 원인을 바깥 소식에서 찾습니다. 그 연결을 뉴스로 이어 줬습니다. 숫자가 변한 날짜 범위를 정하고, 사내에 모아 둔 뉴스 데이터에서 그 기간의 기사를 찾아 사유로 붙입니다. 외부 API를 새로 뚫지 않고 이미 있는 사내 데이터를 API로 불러 썼습니다.

세 방식 중 워크북 XML을 고른 이유

사내 연구에서 화면의 숫자를 AI가 이해하게 할 세 방식을 비교했습니다. GraphRAG는 화면 집계와 원본을 그래프로 잇는 방식인데, 그래프 DB 라이선스 비용이 들고 XML로 같은 문제가 풀려 뺐습니다. 멀티모달은 화면 이미지를 API로 받아 모델에 주는 방식인데, 이미지만으로는 어떤 필터와 조건이 걸렸는지 모델이 알 수 없어 한계가 분명했습니다. 워크북 XML에는 시트마다 쓰는 필드·계산식·필터·집계 방식이 다 들어 있습니다. 이걸 읽어 원본 데이터를 화면 숫자로 바꾸는 계산 규칙을 코드로 만들어 두고, 매일 배치 뒤 그 결과를 XML 구조에 채워 넣습니다. 에이전트는 이 화면의 이 차트에 이 숫자가 이런 기준으로 나온다를 한 번에 읽습니다.

실행 흐름

질문 하나가 증감 사유가 되기까지의 단계입니다. 대시보드 구조와 값은 매일 배치 뒤 미리 적재해 둡니다. 실측 시간·토큰 값은 넣지 않았습니다.

1	**질문 받기** 어느 대시보드의 어떤 숫자가 왜 바뀌었는지 자연어로 묻습니다.	● 완료
2	**화면 찾기** pgvector 검색으로 질문에 맞는 대시보드·시트를 찾습니다.	● 완료
3	**구조와 값 읽기** 워크북 XML 구조에 채워 둔 필드·계산식·필터와 그날 배치로 계산된 값을 읽어 변동을 확인합니다.	● 완료
4	**외부 요인 찾기** 숫자가 변한 날짜 범위를 정하고, 사내에 모아 둔 뉴스에서 그 기간의 기사를 찾습니다.	● 완료
5	**답변** 변동 수치와 그 기간의 뉴스를 함께 묶어 증감 사유로 답합니다.	● 완료

배운 점

가장 크게 배운 것은 AI 프로젝트에서 먼저 볼 것이 모델이 아니라 사람이 실제로 일하는 방식이라는 점입니다. 증감 사유를 찾는 담당자는 데이터가 아니라 뉴스를 봤고, 그걸 그대로 에이전트에 옮기자 답이 쓸모 있어졌습니다. 기술 선택도 같았습니다.

Tableau가 주는 도구, MCP, 그래프 DB처럼 새로 들이는 것보다 고객사가 이미 쓰는 API·DB·배치 위에 올린 구조가 더 빠르고, 고객사에 받아들여졌습니다.